PEARLSAQI — Pakistan AQI Prediction & Forecasting Platform

Complete Project Documentation & Technical Report

Project Report — V4 Final Implementation
Prepared: 6 September 2026

Syeda Warda Rehman
10 Pearls – Data Science Intern

Executive Summary

PEARLSAQI is an end-to-end air-quality intelligence platform built to monitor and forecast Air Quality Index (AQI) across 12 Pakistani cities. The implementation combines historical AQI/pollutant/weather data, feature engineering, an XGBoost regression model, recursive 24/48/72-hour forecasting, SHAP-based explainability, a Flask API, and a React/Vite dashboard. The system also includes automated data-pipeline workflows through GitHub Actions, a DuckDB database, Parquet feature stores, model artifacts, analytics outputs, and scenario simulation.

The project was developed around the supplied requirement of an automated, production-style AQI forecasting system. The final interface provides city exploration, live AQI/pollutant information, a Pakistan map, forecasting, health insights, a custom scenario simulator, and model explainability.

Important deployment status: the documented implementation has a working Flask + React production-style serving path and was exposed during development through a Cloudflare Quick Tunnel. That tunnel is temporary and depends on the local machine. A permanent cloud deployment was investigated, including Gradio/Hugging Face Spaces and Streamlit Community Cloud, but this report does not claim that a permanent cloud deployment was completed. Therefore, the strict “100% serverless” requirement should be considered a deployment-readiness item rather than a completed claim.

Table of Contents

Note: This report uses numbered section headings; Word's Navigation Pane can be used to jump between them.

1. Project Overview

Purpose

PEARLSAQI is an end-to-end air-quality intelligence platform built to monitor and forecast Air Quality Index (AQI) across 12 Pakistani cities. The implementation combines historical AQI/pollutant/weather data, feature engineering, an XGBoost regression model, recursive 24/48/72-hour forecasting, SHAP-based explainability, a Flask API, and a React/Vite dashboard. The system also includes automated data-pipeline workflows through GitHub Actions, a DuckDB database, Parquet feature stores, model artifacts, analytics outputs, and scenario simulation.

The project was developed around the supplied requirement of an automated, production-style AQI forecasting system. The final interface provides city exploration, live AQI/pollutant information, a Pakistan map, forecasting, health insights, a custom scenario simulator, and model explainability.

Important deployment status: the documented implementation has a working Flask + React production-style serving path and was exposed during development through a Cloudflare Quick Tunnel. That tunnel is temporary and depends on the local machine. A permanent cloud deployment was investigated, including Gradio/Hugging Face Spaces and Streamlit Community Cloud, but this report does not claim that a permanent cloud deployment was completed. Therefore, the strict "100% serverless" requirement should be considered a deployment-readiness item rather than a completed claim.

Problem Statement

Air pollution changes over time and differs substantially between Pakistani cities. A useful operational system therefore needs more than a single AQI number: it should ingest current conditions, preserve historical data, transform raw observations into model-ready features, forecast future AQI, expose predictions through an accessible dashboard, and provide explanations that help users understand the drivers behind predictions.

Project Objectives

The implementation targets five connected objectives: (1) build a reliable AQI data pipeline; (2) create a reusable feature store and historical training dataset; (3) train and evaluate a production forecasting model; (4) expose live and forecast information through a web dashboard; and (5) add explainability, scenario analysis, alerts, and city-level analytics.

2. Requirements Interpretation

Original project direction

The supplied brief calls for an end-to-end AQI forecasting pipeline with automated data collection, feature engineering, historical backfill, model training/evaluation, automated CI/CD, a dashboard, EDA, explainability, alerts, and multiple forecasting approaches. It also describes a "100% serverless stack."

Implementation interpretation

Where the brief offered alternatives, the project selected practical equivalents rather than adding every named technology. GitHub Actions was selected for automation instead of Airflow/n8n; Open-Meteo is the current data source rather than AQICN/OpenWeather; DuckDB + Parquet are used for persistent data/feature storage; XGBoost is the production model; and React + Flask form the final dashboard/API stack.

Compliance matrix

Requirement	Implementation	Status
AQI forecasting	XGBoost V4 production model with recursive 24/48/72-hour forecasting	Completed
Historical dataset	13,068 observations across 12 cities; 2023-01-08 to 2025-12-31	Completed
Feature engineering	108 production model features; pollution/weather/time relationships	Completed
Model evaluation	MAE, RMSE, R^2 plus error/category/horizon analysis	Completed
Interactive dashboard	React + Vite frontend with Flask API	Completed
Explainability	SHAP/LIME integration; 108-feature SHAP view	Completed
Scenario analysis	Custom pollutant/weather what-if simulator	Completed
Automation	GitHub Actions hourly data-pipeline workflow	Implemented
Database / feature store	DuckDB + Parquet feature store	Implemented
Alerts	AQI alert functionality integrated in backend	Implemented
Permanent serverless hosting	Cloud deployment investigated; Quick Tunnel used for temporary public access	Not yet completed
Hopsworks / Vertex AI	Not used in the final implementation	Alternative not required by current architecture
Airflow / n8n / MLflow	Not used in the final implementation	GitHub Actions selected instead
AQICN / OpenWeather	Not used as the current source	Open-Meteo selected

3. System Architecture

High-level architecture

The system can be understood as six layers: external data sources → ingestion/cleaning → feature engineering/storage → model training/registry artifacts → Flask API → React dashboard. GitHub Actions provides scheduled automation around the data and database layer. The frontend requests API data through relative `/api` routes in development and the Flask server can serve the built React application in production.

Data flow

Raw weather and air-quality information is collected, normalized, transformed into time/pollution/weather features, and persisted in the project's database/feature-store assets. Historical analytics and prediction-analysis tables support model evaluation. The production XGBoost model consumes the engineered feature vector, while the forecast engine recursively produces the 24/48/72-hour horizons. The Flask layer exposes predictions, trends, statistics, manual/scenario predictions, and explainability data to the UI.

Key storage artifacts

The implementation uses `database/aqi.duckdb` for database storage and Parquet feature stores including `data/feature_store/historical_features.parquet` and `data/feature_store/live_features.parquet`. The production model is stored at `models/benchmark/v4/final_candidate/xgboost_v4_final_108.pkl` and contains the final 108-feature XGBoost regressor.

4. Data Pipeline & Feature Engineering

Data coverage

The EDA dataset contains 13,068 daily observations across 12 cities, covering 2023-01-08 through 2025-12-31. Each city contributes 1,089 recorded days in the analyzed historical dataset.

Cities

Bahawalpur, Faisalabad, Gujranwala, Hyderabad, Islamabad, Karachi, Lahore, Multan, Peshawar, Quetta, Rawalpindi, and Sialkot.

Variables

The analyzed daily dataset contains city identifiers/names, date, AQI, PM2.5, PM10, NO2, SO2, CO, O3, temperature, humidity, precipitation, windspeed, and AQI category. The production model uses a larger engineered feature space totaling 108 features.

Feature engineering

The project uses time-based and derived relationships so the model can capture temporal patterns, pollutant interactions, weather effects, lagged/rolling behavior, pollution ratios/sums, and other engineered signals. The live feature store reports 111 columns, while the serving model expects exactly 108 model features.

Data quality

The EDA notebook reports zero missing values in the analyzed 13,068-row daily dataset. It also verifies 12 unique cities and a continuous date field after conversion/sorting. This supports a clean baseline for model evaluation, while production monitoring remains important because live API inputs can change.

5. Exploratory Data Analysis

EDA workflow

The supplied `eda.ipynb` loads `analytics/daily_aqi.csv`, converts dates to datetime, sorts by city/date, and performs distribution, city-comparison, trend, correlation, pollutant, category, monthly, prediction-error, model-performance, forecast-horizon, and data-quality analyses.

Core findings

Historical AQI ranges from 44 to 538, with a mean of 137.61 and median of 138.0. Lahore has the highest average AQI at 175.88, while Quetta has the lowest average at 83.30. PM2.5 has the strongest correlation with AQI (0.8157), followed by PM10 (0.6456), CO (0.6042), SO2 (0.5923), NO2 (0.4868), and O3 (0.2215).

City ranking

Average AQI ranking in the EDA is: Lahore (175.88), Faisalabad (175.08), Gujranwala (163.07), Multan (156.73), Sialkot (156.27), Islamabad (136.14), Rawalpindi (136.14), Bahawalpur (134.30), Peshawar (133.56), Karachi (102.72), Hyderabad (98.14), and Quetta (83.30).

Prediction analysis

The prediction-analysis dataset contains 2,616 evaluated prediction rows. Overall performance reported by the notebook is MAE 13.416, RMSE 18.795, and R^2 0.844. Error analysis is also broken down by city, AQI category, and time period.

Category-specific error

Mean absolute error is higher in the most extreme AQI categories: Hazardous 32.98, Very Unhealthy 22.59, Unhealthy 12.63, Unhealthy for Sensitive Groups 12.49, and Moderate 12.32. This indicates that extreme conditions are harder for the model to predict accurately and should be treated carefully in deployment.

6. Machine Learning Model

Model choice

The final serving model is an XGBoost regression model saved as `xgboost_v4_final_108.pkl`. XGBoost was selected as the production candidate because the problem is a nonlinear tabular regression task with interactions between pollution, weather, and temporal features.

Model verification

The saved artifact was verified by loading it with joblib and confirming the object type as `xgboost.sklearn.XGBRegressor` and successful model loading. The serving layer is configured around the 108-feature production artifact.

Evaluation

The EDA notebook compares multiple model/variant rows through `analytics/model_performance.csv` and evaluates production predictions using MAE, RMSE, R^2, within-error metrics, error distributions, city-level error, and category-level error. The production evaluation shown in the notebook is MAE ≈13.42, RMSE ≈18.79, R^2 ≈0.844 over 2,616 predictions.

Forecast horizons

The forecast-history analysis contains 396 forecast records, with 324 rows having actual AQI values. Horizon performance is: Day 1 MAE 11.13 / RMSE 14.56 / bias -0.06; Day 2 MAE 23.23 / RMSE 30.48 / bias +7.99; Day 3 MAE 31.13 / RMSE 38.17 / bias +16.30. The expected degradation with longer recursive horizons is therefore measurable and should be communicated to users.

7. Backend / API

Framework

Flask is used as the serving backend. CORS support is included for frontend/API interaction. The backend loads the production model, live feature store, configuration, alerts, model registry information, and explainability components.

Routes

The documented backend exposes `/`, `/health`, `/predict/<city>`, `/api/trends/<city>`, `/api/model-stats`, `/api/manual-predict`, `/api/scenario-predict`, and an `/api/predict/<city>` alias. The root path can serve the built React frontend while API routes remain available for the dashboard.

Health endpoint

The tested health response reports status `ok`, model `XGBoost V4 Final 108`, 108 model features, 111 live columns, live feature-store status, 15,696 live rows, and PM2.5 pollution-ratio availability.

Production serving

The Flask server is configured for host `0.0.0.0`, uses the platform-provided `PORT` environment variable with 5000 as the default, and runs with debug/reloader disabled for production-style serving.

8. Frontend / Dashboard

Technology

The final dashboard is a React 19 + Vite application. It uses React Leaflet for the map, Recharts for charts, and Lucide React for interface icons. The frontend communicates with the backend through relative `/api` routes, which avoids hard-coding the local development host into production requests.

Main views

The sidebar provides Overview, Cities, Scenario Lab, Pakistan AQI, AQI Forecast, Air Quality Insights, and Model Explainability. The interface is designed around a dark PEARLSAQI visual identity and presents the most important operational information without requiring users to inspect raw model files.

Overview

The overview screen presents Karachi's current AQI, pollutant cards, weather conditions, monitored cities, and forecast context. The shown example contains AQI 60 with PM2.5 14, PM10 27, NO2 9, SO2 5, CO 190, and O3 42.

Cities

The Cities screen provides a visual directory of the 12 monitored locations. Users can select a city and inspect its live air-quality status.

Forecast

The AQI Forecast screen presents 24/48/72-hour predictions and model-driven forecast insights.

Map

The Pakistan AQI page uses an interactive Leaflet map with city markers and AQI category information.

Insights

The Insights page combines pollutant breakdown, health recommendation, city comparison, city directory, and live AQI ranking.

9. Scenario Simulator

Purpose

The Custom Scenario Simulator provides a what-if analysis interface. Users can change pollution and weather conditions and request a scenario prediction rather than waiting for the next live update.

Inputs

The interface exposes PM2.5, PM10, O3, NO2, SO2, CO, temperature, humidity, and wind speed controls. Quick scenarios include Custom, Heavy Smog, Traffic Rush, After Rain, and Hot Afternoon.

API integration

Scenario prediction is routed through the Flask API. The React/Vite development proxy maps `/scenario-predict` to the backend so the UI does not depend on a hard-coded localhost port.

10. Explainability & Analytics

SHAP

The Model Explainability view presents SHAP feature contributions for Day 1, Day 2, and Day 3. The interface reports 108 features and distinguishes AQI-increasing and AQI-decreasing contributions.

LIME

LIME is included in the project's explainability tooling alongside SHAP. SHAP is the primary visual explanation shown in the final dashboard because it fits the feature-contribution presentation used by the serving model.

Analytics assets

The project contains analytics outputs for AQI categories, city comparison, city statistics, daily AQI, monthly AQI, pollutant statistics, model performance, and prediction analysis. These support both EDA and the dashboard's analytical narrative.

11. Automation & CI/CD

GitHub Actions

The project uses GitHub Actions for scheduled pipeline automation. The intended workflow is hourly feature/data updates, with training/update operations supported by the project architecture. This avoids requiring a continuously running local scheduler.

Workflow debugging

A YAML syntax error was encountered around the database-summary step. The nested Python/heredoc construction caused GitHub to report an invalid workflow on line 236. The summary step was simplified to a quoted `python -c` block that opens DuckDB, queries the `aqi` and `weather` tables, prints min/max date, row count, and city count, then closes the connection.

Operational lesson

For CI workflows, simpler shell/Python boundaries are safer than deeply nested heredocs. YAML indentation, shell quoting, and Python triple-quoted SQL need to be validated together because an error in one layer can appear as a YAML syntax error in another.

12. Git, Version Control & Repository Work

Repository management

The project is maintained in Git with a GitHub `main` branch. Changes were committed in logical steps, including serving the React build from Flask and maintaining README/screenshot assets.

Database conflict

A binary `database/aqi.duckdb` conflict occurred during rebasing because automated database updates had diverged from local history. The resolution used the remote version of the database, staged it, and continued the rebase. This prevented a binary merge conflict from corrupting the repository history.

Screenshot asset issue

The README screenshot directory was temporarily removed, causing GitHub README images to display as broken links. The recovery approach was to restore the `screenshots/` directory from the previously successful screenshot commit, then commit and push the restored assets.

Filename handling

Two screenshot filenames contain spaces: `custom simulator.png` and `pak map.png`. When referenced from GitHub Markdown, the spaces should be URL-encoded as `%20` to avoid broken image paths.

13. Problems Faced & Solutions

Problem: local API URLs

The frontend initially used hard-coded localhost endpoints for some calls. Solution: route requests through relative `/api` paths and configure the Vite proxy for local development. This makes the frontend portable between local development and Flask-served production.

Problem: Flask/React deployment integration

The frontend and backend originally behaved as separate development servers. Solution: Flask was updated to serve the built React `dist/index.html` and static assets, with a fallback route for client-side navigation.

Problem: Git rebase / binary DB conflict

DuckDB is a binary file and is not line-mergeable. Solution: explicitly choose the intended database version during the rebase, stage it, and continue the rebase.

Problem: GitHub Actions YAML

The workflow became invalid around a database-summary block. Solution: simplify the embedded Python invocation and avoid fragile nested heredoc syntax.

Problem: README screenshots disappearing

Removing the screenshot directory broke every README image even though the Markdown remained present. Solution: restore the directory from a known-good Git commit and verify filenames and file sizes before pushing.

Problem: screenshot replacement not detected

Git did not create a commit when a screenshot was visually expected to have changed. Git tracks file content, not modification timestamps. If the PNG bytes are identical, `git status` correctly reports no change. The correct fix is to replace the file with genuinely different image content and then verify the Git diff/status.

Problem: temporary public access

Cloudflare Quick Tunnel successfully exposed the local Flask app, but the URL depends on the local server and tunnel process. Solution for production is a real cloud-hosted runtime rather than treating the Quick Tunnel as permanent deployment.

14. Deployment Status & Serverless Readiness

Current working deployment path

The project can be run locally as Flask + React and was successfully exposed through a Cloudflare Quick Tunnel during development/testing. This demonstrates external accessibility but is not permanent hosting.

Why Quick Tunnel is not final deployment

A Quick Tunnel forwards traffic to the developer's machine. If the machine, Flask server, or tunnel stops, the public application stops. Therefore it should be documented as a testing/demo mechanism, not as a serverless production deployment.

Gradio/Hugging Face option

Gradio on Hugging Face Spaces was identified as a free hosting route that aligns with the original brief's explicit mention of Gradio. To use it, the project would need a Gradio entry application and a cloud-compatible packaging of the model/data dependencies. This is a deployment option, not a claim that the current React UI has already been migrated to Gradio.

Streamlit Community Cloud option

Streamlit Community Cloud was also investigated as a free GitHub-connected hosting option. The existing React dashboard is not automatically a Streamlit app, and the current `dashboard.py` expects a local API, so a direct cloud deployment would require either adapting the Streamlit app to cloud-compatible data/model access or hosting the backend separately.

Readiness conclusion

The software architecture is deployment-ready in the sense that Flask can serve the built frontend and uses environment-based port configuration. The remaining step for strict 100% serverless compliance is to host the runtime and persistent data/model dependencies on a cloud service that does not depend on the developer PC.

15. Testing & Validation

Build validation

The React application was built successfully with Vite. The build produced a CSS bundle and a JavaScript bundle; Vite emitted a chunk-size warning because the main JS bundle is above 500 kB, but the build itself completed successfully.

Model validation

The production XGBoost artifact was loaded successfully using joblib and verified as an `XGBRegressor`. The health endpoint also validates that the serving model expects 108 features and that the live feature store is loaded.

Data validation

The EDA notebook confirms 13,068 rows, 12 cities, no missing values in the analyzed daily dataset, and consistent per-city historical coverage.

Forecast validation

Forecast history contains 396 records across horizons 1–3; 324 have actual AQI values available for evaluation. Horizon-specific MAE/RMSE/bias were computed to quantify recursive forecast degradation.

16. Project Structure

Major directories

A representative repository layout is:
`app.py` — Flask backend;
`frontend/` — React/Vite application;
`src/` — configuration, feature/model/data utilities;
`data/feature_store/` — historical/live Parquet features;
`database/aqi.duckdb` — DuckDB database;
`models/benchmark/v4/final_candidate/` — production model artifact;
`analytics/` — EDA/evaluation CSV outputs;
`notebooks/eda.ipynb` — exploratory analysis;
`.github/workflows/` — automation;
`screenshots/` — README/dashboard screenshots;
`Docker*` / deployment files — container support where present.

Frontend dependencies

The final package uses React 19, React DOM 19, Leaflet 1.9, React Leaflet 5, Recharts 3, and Lucide React, with Vite 8 and the React plugin for builds.

17. Design Decisions

Why XGBoost

Strong fit for nonlinear tabular data and engineered feature interactions; simple artifact loading through joblib.

Why DuckDB + Parquet

Local analytical database plus columnar feature-store files provide a lightweight, reproducible data layer without introducing unnecessary cloud infrastructure.

Why GitHub Actions

Provides scheduled automation directly from the repository and satisfies the brief's allowance for GitHub Actions as an Airflow alternative.

Why React + Flask

The final UI requires a richer multi-page visual experience than a basic data app, while Flask provides a compact Python API around the ML model.

Why SHAP

Feature-level contribution views make the model's decisions inspectable rather than presenting forecasts as unexplained numbers.

Why scenario simulation

It turns the model into an interactive what-if tool, allowing users to explore the effect of changed pollution/weather conditions.

18. Limitations & Risks

Forecast uncertainty

Recursive forecasts degrade with horizon: the evaluated MAE rises from 11.13 at horizon 1 to 31.13 at horizon 3. Users should treat longer forecasts as increasingly uncertain.

Extreme AQI conditions

The EDA shows substantially higher errors for Hazardous and Very Unhealthy observations. The model should not be treated as equally reliable across all AQI ranges.

Live-data dependency

Live predictions depend on the availability and quality of upstream weather/AQI data. API changes, outages, or abnormal inputs can affect predictions.

Cloud deployment

A permanent cloud deployment is still required for strict serverless compliance. A local Quick Tunnel should not be treated as the final hosting solution.

Model drift

Air-quality relationships can change by season, emissions, weather, and data-source behavior. Automated retraining and drift monitoring should be strengthened over time.

Bundle size

The React build produced a large JavaScript bundle. Code splitting/lazy loading could improve initial load performance.

19. Future Improvements

Cloud deployment

Deploy the complete runtime on a free/low-cost cloud platform that supports the chosen interface and model artifacts.

Model improvements

Compare stronger baselines, direct multi-horizon models, gradient boosting variants, and possibly deep-learning/statistical approaches if justified by data volume.

Forecast calibration

Add uncertainty intervals or prediction bands so users can see confidence rather than a single point forecast.

Monitoring

Add systematic data-drift, feature-drift, prediction-drift, and model-performance monitoring.

Automation hardening

Separate hourly ingestion from daily/weekly retraining jobs, add retry logic, workflow tests, and explicit artifact versioning.

Frontend optimization

Lazy-load heavy chart/map components and reduce initial JS bundle size.

Reproducibility

Pin dependency versions, document environment variables, add automated tests, and provide a one-command deployment workflow.

20. Conclusion

Final assessment

PEARLSAQI is a substantial end-to-end AQI forecasting system rather than only a machine-learning notebook. It connects data ingestion/storage, feature engineering, model training/evaluation, forecasting, API serving, a multi-view dashboard, explainability, scenario simulation, analytics, and scheduled automation.

What was achieved

The final implementation supports 12 Pakistani cities, a 108-feature XGBoost serving model, 24/48/72-hour forecasting, SHAP explanations, live-data integration, city comparison, an interactive Pakistan map, health guidance, scenario analysis, and automated pipeline infrastructure.

Deployment caveat

The remaining major gap relative to the original “100% serverless” wording is permanent cloud hosting. The current code is structured for deployment, but the development Quick Tunnel is not equivalent to a serverless hosted service. This should be resolved before making a final claim of full serverless compliance.

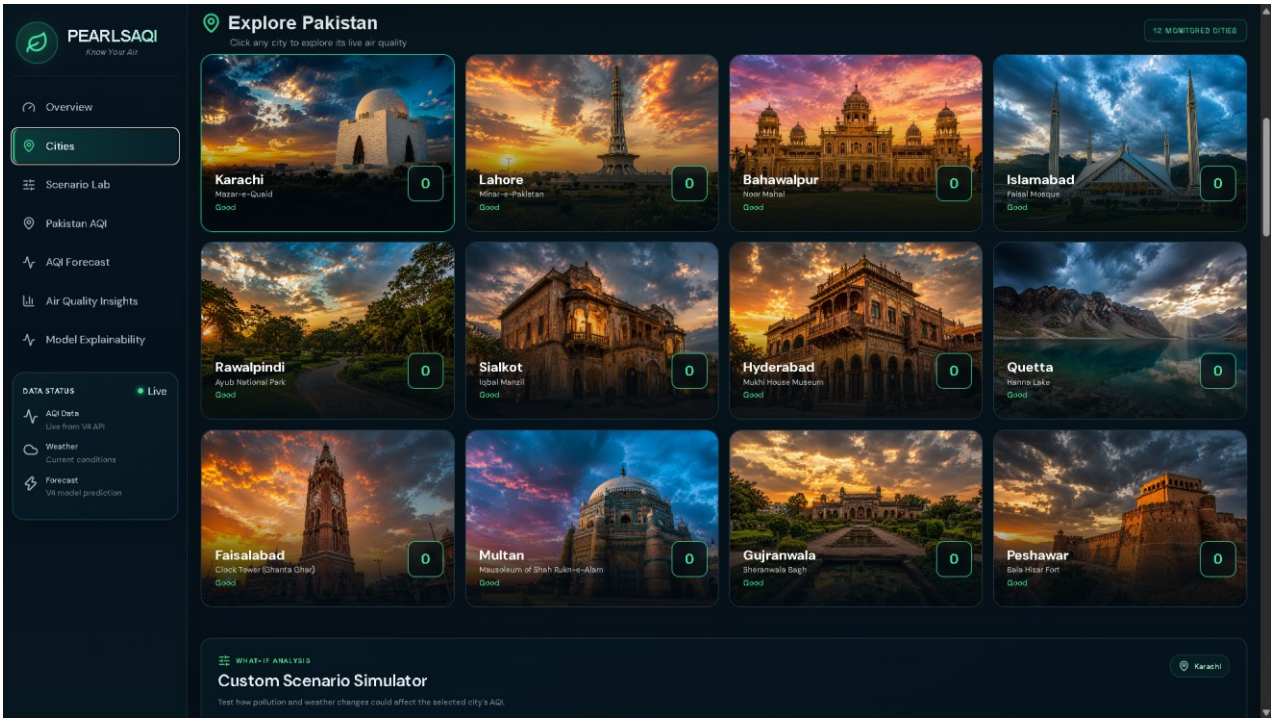
Appendix A — Final Dashboard Screenshots

The following screenshots document the final user-facing interface reviewed for this report.

Overview Dashboard

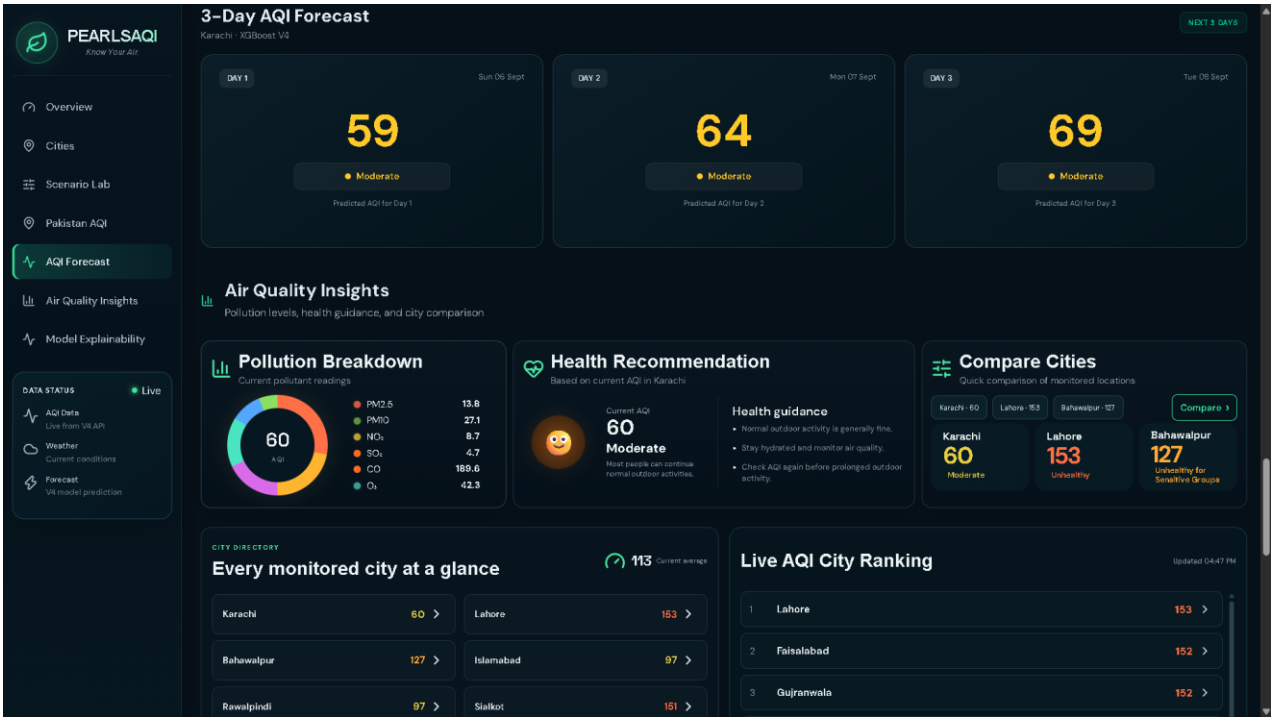
Live Karachi AQI, pollutant cards, weather conditions, monitored cities, and forecast context.

Cities Dashboard



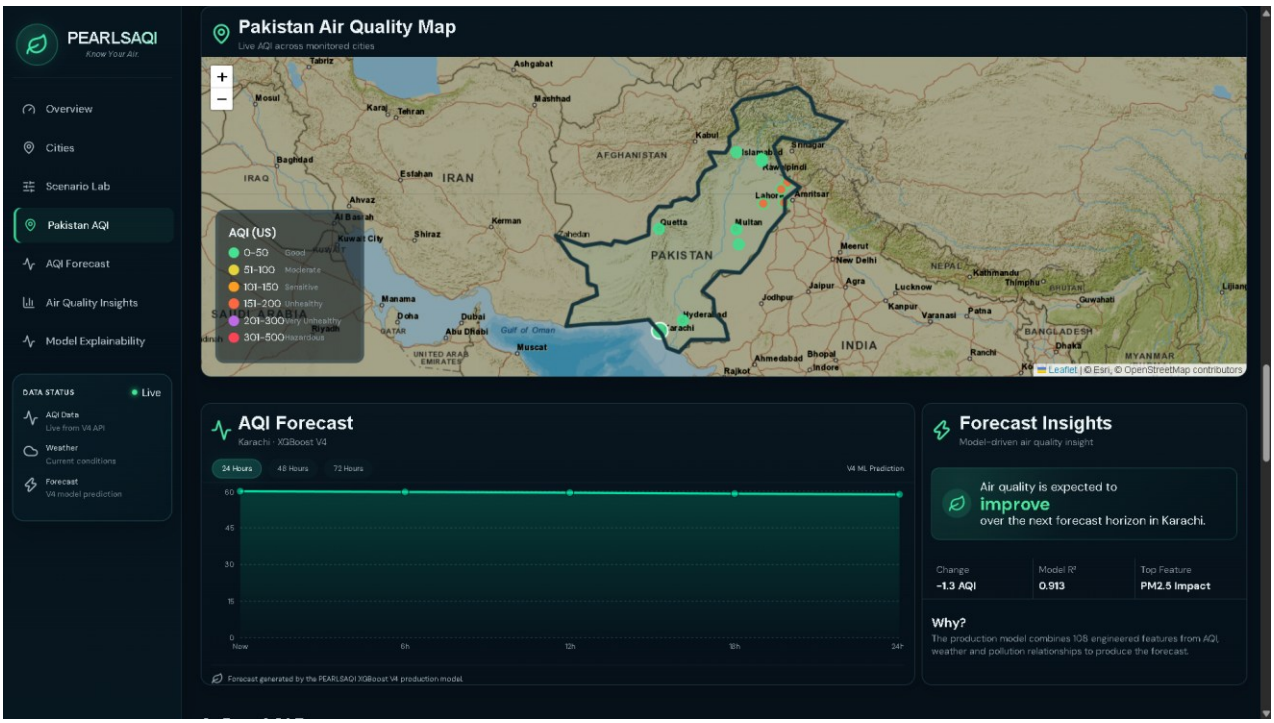
12 monitored Pakistani cities with location imagery, AQI status, and city selection.

3-Day AQI Forecast



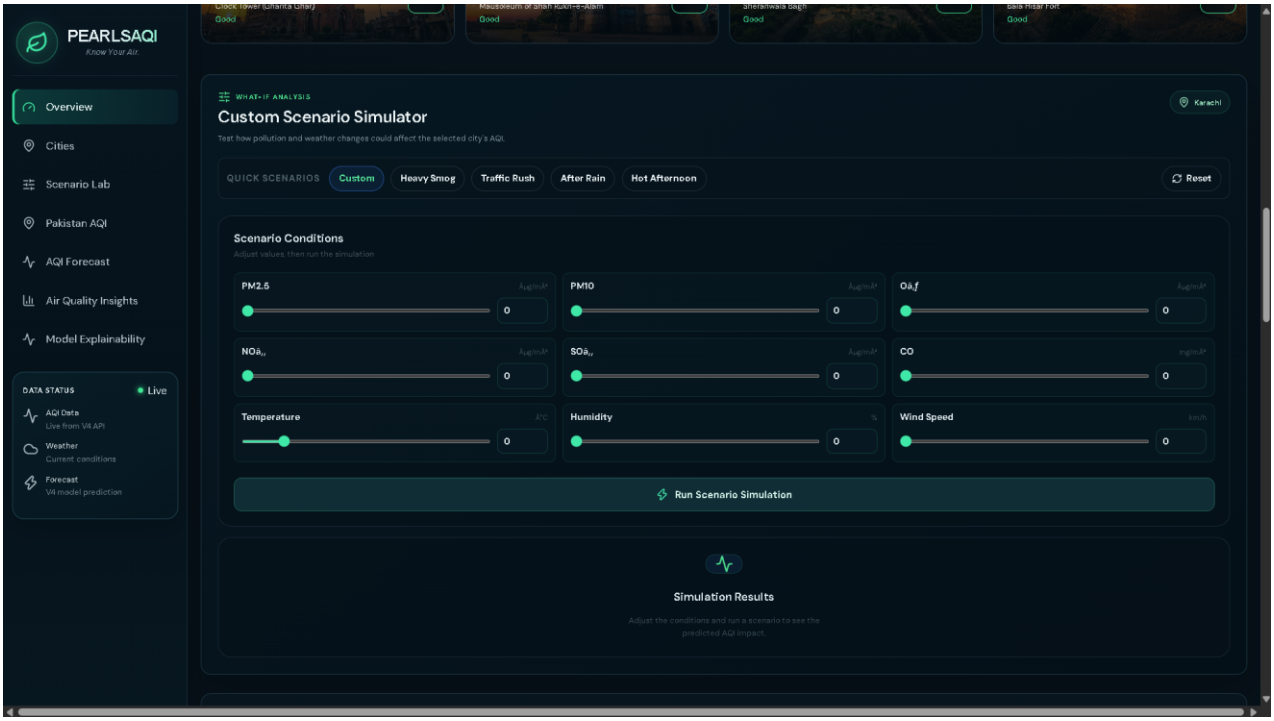
Day 1/Day 2/Day 3 predictions with air-quality insights and model context.

Pakistan AQI Map



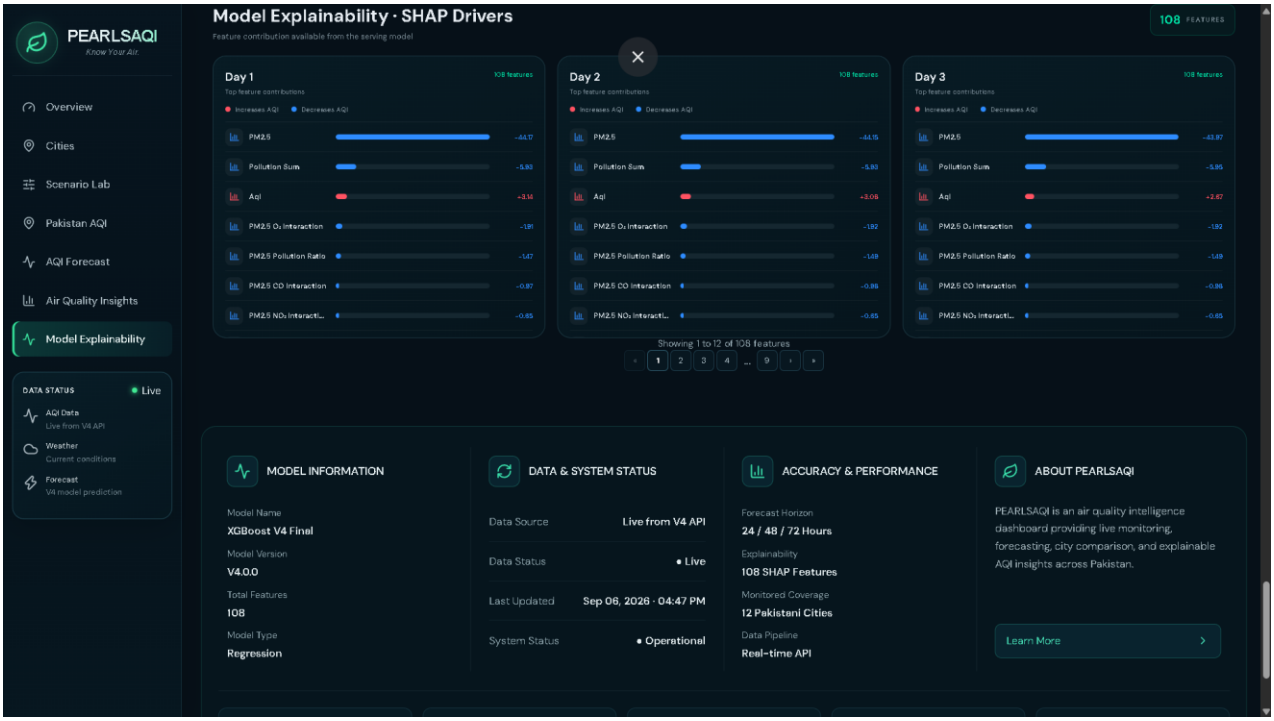
Geographic view of monitored-city AQI values and forecast panel.

Custom Scenario Simulator



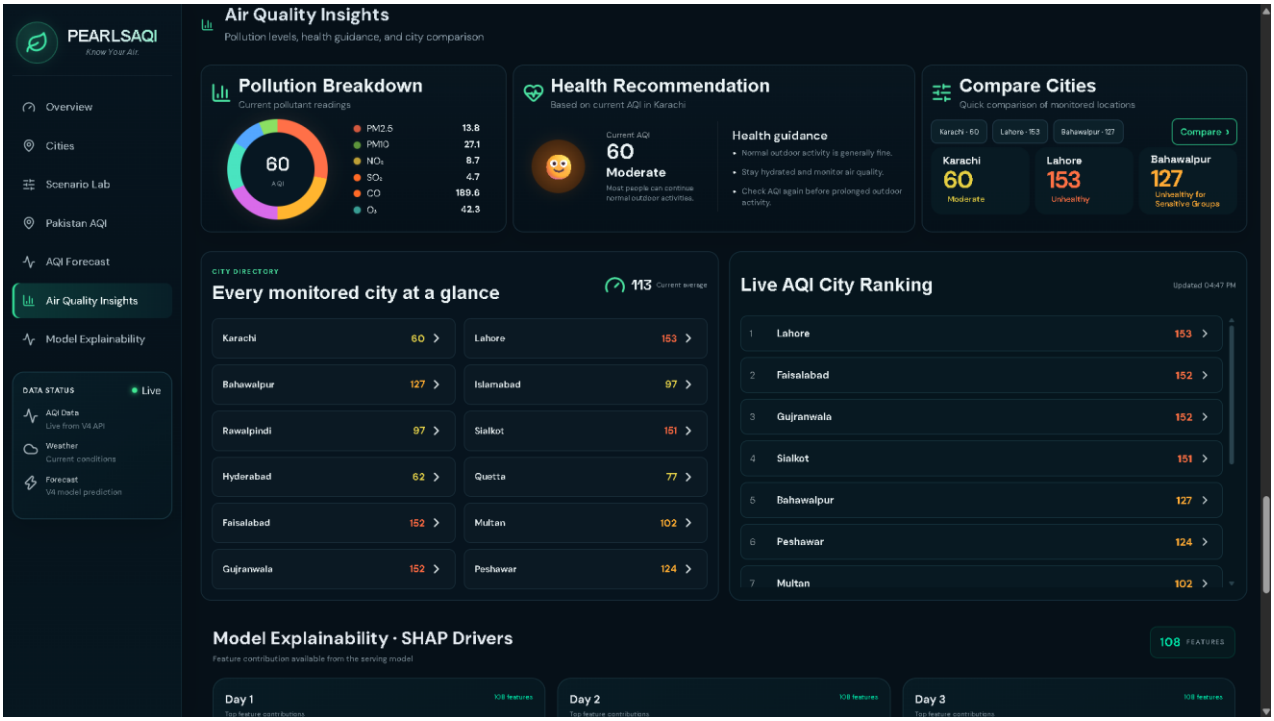
What-if controls for pollutants and weather variables followed by a simulation result area.

Model Explainability



SHAP driver cards for three forecast days and model metadata.

Air Quality Insights



Pollution breakdown, health recommendation, city comparison, directory, and live AQI ranking.

Appendix B — Key Metrics Snapshot

Metric	Value
Historical observations	13,068
Cities	12
Historical date range	2023-01-08 → 2025-12-31
Historical AQI range	44 → 538
Historical mean AQI	137.61
Strongest pollutant correlation	PM2.5 = 0.8157
Production evaluated predictions	2,616
Overall MAE	13.416
Overall RMSE	18.795
Overall R^2	0.844
Forecast-history records	396
Forecast records with actual AQI	324
Horizon 1 MAE	11.126
Horizon 2 MAE	23.233
Horizon 3 MAE	31.132
Production model features	108
Live feature-store columns	111

End of report.